

Proactive Intervention for Harmful Online Content: A Fast-Slow Dual-Stage Framework Integrating Local Models and Large Language Models

Zixuan Qi

High School Student

Shanghai Nanyang Model High School, Shanghai, China

19901636898@163.com

Abstract— Online content governance has mostly relied on post-hoc moderation, which is slow and gives users little interaction. Large language models (LLMs) can understand nuanced semantics, but calling them directly for every high-frequency, real-time input is too slow and expensive. We present Shouwang, a browser plugin that checks a message before it is posted. It uses a fast-slow dual-stage architecture: a lightweight local MacBERT gatekeeper screens every draft quickly, and only suspicious texts are escalated to an LLM for deeper analysis and interactive persuasion. To adapt the local detector to how Chinese is actually written online, we built “Between the Line”, an offensive-language dataset of 2,414 samples in 16 categories, about 20% of which use complex expressions (abbreviations, homophones, irony). We train the model jointly on this dataset and the public benchmarks COLD and weibo_senti_100k. The best gatekeeper reaches F1 0.886 (mean of three seeds) on our custom test set and 0.882 on a unified mixed test set, about 92% above SnowNLP’s 0.46 there, at about 89% overall accuracy. Code, model, and dataset are released as open source.

Keywords— large language model; content governance; proactive intervention; offensive language detection; text sentiment analysis

I. Introduction

Most online communication now happens on social platforms, whose anonymity and openness also host cyberbullying, offensive speech, and misleading information. Because language carries these interactions, catching and intervening on risky text in time has become central to online governance.

Existing platform mechanisms are mostly post-hoc. Moderation starts only after publication, so harmful content can spread before anyone handles it. Punitive measures such as removal, blocking, or throttling leave little room for dialogue with the user. Fixed moderation notices rarely explain what was wrong, and they cannot guide a user before they post. This one-way, punishment-oriented governance gives people little chance to self-correct.

Large language models understand and generate language with nuance, which opens a different route for content governance. Prior work shows that language models can parse and structure user reviews [1], deep learning can capture sentiment in web comments [2], and combining topic and sentiment analysis helps expose deceptive or noisy reviews [3]. Yet most applications use these as analysis tools after publication, not as real-time, conversational advisors. If the gate moved to just before posting and an LLM talked with the user, governance could shift from “penalize after detection” to “identify and guide before posting”.

Using an LLM to check every keystroke is not practical. From capturing text on the front end to receiving the model’s verdict takes several seconds, and during that time the user may revise the text many times. Calling the model on every change queues up requests, breaks real-time stability, and most of the cost goes to text that is perfectly normal. A purely LLM-based detector therefore cannot serve high-frequency screening. Our initial probe also found that SnowNLP [4], a dictionary-based Chinese sentiment tool, only reaches an F1 of 0.46 on our unified test set, too low to rely on.

We therefore combine a local lightweight model (fast screening), an LLM (deep judgment), and an interactive popup (guidance) into one fast-slow collaborative framework. Our contributions are: (i) moving the intervention point from post-

publication review to pre-publication guidance, so users are guided instead of merely punished; (ii) a dual-stage architecture that balances speed, detection quality, and cost; (iii) “Between the Line”, a dataset of modern Chinese internet expressions; and (iv) a gatekeeper trained through three groups of comparative experiments on data, model, and training settings.

II. Related Work

A. Offensive Language and Hate Speech Detection

Early detection treated harmful content as a single label, but Davidson et al. [8] showed that hate speech and ordinary offensive language must be separated, and released a widely used benchmark. In Chinese, Deng et al. [5] built COLD, 37k comments spanning racial, gender, and regional bias, with the COLDdetector baseline. Lu et al. [6] proposed ToxiCN, whose fine-grained taxonomy distinguishes direct from indirect toxicity and hate speech from offensive language. Jiang et al. [7] released SWSR, a Sina Weibo sexism dataset with a 3,016-term lexicon, showing that Chinese harmful language is often implicit and lexicalized. On the model side, Mozafari et al. [9] showed that fine-tuning BERT raises hate speech detection. These works validate pre-trained encoders and language-specific benchmarks, yet they all classify text that has already been published.

B. LLM-Based Content Moderation

Large language models offer an alternative route. Kumar et al. [10] evaluated commodity LLMs (GPT-3.5/4, Gemini Pro, Llama) for toxicity detection and found they often outperform dedicated classifiers, although they lag in high-frequency, rule-driven settings.

C. Gap: Pre-Publication Intervention

Gorwa et al. [11] framed algorithmic content moderation as a platform-governance problem, yet deployed systems remain overwhelmingly post-hoc. To our knowledge no peer-reviewed work studies real-time pre-publication intervention that both detects risky text and guides the user. Shouwang targets this gap: a lightweight gatekeeper screens every draft,

and only suspected text is escalated to an LLM for depth and dialogue.

III. System Design

A. Overall Architecture

Shouwang is a Manifest V3 Chrome extension with a local Python (FastAPI) backend: a front-end capture layer (content.js), a decision layer (app.py, linking the local model and the LLM), and an interaction layer that renders popups. The backend runs on the user's own machine at 127.0.0.1:8000, so drafts are screened on-device and only flagged drafts leave the device. The current build targets Weibo comment boxes (weibo.com). The pipeline is shown in Fig. 1; a popup with a real dialogue is shown in Fig. 2.

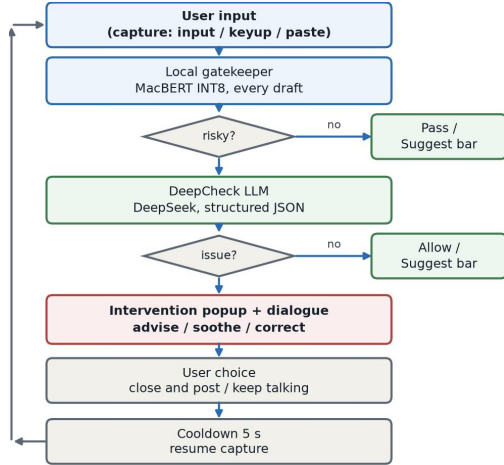


Fig. 1. System overview of the fast-slow proactive intervention pipeline.

B. Front-End Content Capture

The front end listens to input, keyup, and paste events on the comment box and streams the current text to the backend in real time. Every detection starts from this stream.

C. Local Fast Gatekeeper

We fine-tune a MacBERT model [12] (hfl/chinese-macbert-base, whole-word masking) as the local gatekeeper and compare it with BERT [13] and SnowNLP [4]. Since the gatekeeper is the first-stage “doorman” rather than the final judge, missing a malicious sentence means the whole system never sees it, whereas over-blocking only costs a few tokens. We therefore prioritize recall, while still tracking precision and F1. After export and INT8 quantization, the model file shrinks by 75%, inference latency drops by 38%, and loading becomes roughly ten times faster, with accuracy unchanged.

Intervention is graded: none, suggest (a non-blocking bar), and warn (a popup). The intervention decision fuses two signals: (1) an attack lexicon hit for explicit profanity, and (2) the local model’s offense score, which covers homophones and abbreviations a lexicon cannot enumerate. The score is binarized at a calibrated threshold of 0.5: a lexicon hit or a score above the threshold is treated as an attack and raises a warn popup, whereas negative sentiment without an attack signal (from the lexicon or SnowNLP) only raises a suggest bar. This yields the semantics of “complaints get a bar, attacks get a popup”: “it’s so annoying that it’s raining today” scores

0.249 and gets a bar, while “the service in this shop is really bad” scores 0.627, above the threshold, and is treated as an attack.

D. LLM Integration and Prompts

Suspected texts are sent to a remote LLM (DeepSeek). A built-in prompt casts the model as a “careful Chinese online-speech intervention assistant” that reviews the draft for both emotional/offensive content and factual errors, and replies warmly in the second person. It must return strict JSON so the program can parse it automatically, including fields for emotion label, intensity, tone, intervention mode, factual-error and hate-speech flags, whether intervention is needed, the intervention level (none/suggest/warn), and the AI reply text. A separate persuasion prompt injects the last ten turns of context, the intercepted original text, and the judged reason, so the dialogue addresses this specific draft instead of issuing a generic warning.

E. Intervention Popup

The popup appears when the local model or the LLM identifies clear risk. It presents the detected emotion and level on top and hosts a multi-turn dialogue in the center: the LLM talks the user out of offensive phrasing, calms excessive negativity, or points out factual mistakes. Because false alarms are inevitable, the user can always close the popup and post anyway, and a cooldown mechanism stops the popup from reappearing within a short window.



Fig. 2. Intervention popup with a real LLM dialogue (a user reframing a venting post).

IV. Model: BERT and MacBERT

BERT [13] is a stack of Transformer encoder layers. Each layer applies multi-head self-attention over the sequence of token embeddings, then a position-wise feed-forward network. Because attention lets every token attend to its full context in both directions, BERT captures bidirectional context instead of reading left to right. It is pre-trained with two objectives: masked language modeling (MLM), which predicts masked tokens from their surrounding context, and next-sentence

prediction (NSP), which decides whether two sentences are naturally adjacent. Pre-training on large unlabeled corpora yields a general-purpose encoder that we then fine-tune. For Chinese, bert-base-chinese tokenizes at the character level and stacks 12 layers with 768 hidden units and 12 attention heads, roughly 110M parameters.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V \quad (1)$$

$$\text{head}_i = \text{Attention}(QW_{iQ}, KW_{iK}, VW_{iV}) \quad (2)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (3)$$

We refine BERT with MacBERT [12], a Chinese pre-trained model. It applies whole-word masking (WWM): instead of masking single characters or sub-tokens, it masks all characters of a segmented word, so the model must recover word-level units. Rather than a fixed [MASK] token, MacBERT fills masked positions with a similarly-meaning word, turning MLM into a soft correction task, and it drops the NSP objective. These changes suit Chinese, where the same character behaves differently inside different words.

We choose MacBERT as the gatekeeper because it best matches our input, short colloquial Chinese internet text in which word boundaries carry meaning. In our experiments it reaches an F1 of 0.896, above BERT’s 0.881 and far above the untrained baseline of 0.479 (Section VI), and its compact size keeps inference fast enough for the first stage an LLM cannot serve.

V. Dataset: Between the Line

Public datasets mostly predate modern Chinese internet habits, so they cover homophones, abbreviations, implied mockery, and irony poorly. We therefore built our own dataset, “Between the Line”. A survey of 70 respondents (67 valid, consenting responses) asked each to recall the most unbearable online remark they had seen and, as a control, a remark everyone would find normal. Frequently mentioned types (gender antagonism, regional discrimination, personal attacks, sexual innuendo, sarcasm, internet memes, manipulative preaching, and trolling) drove our category design. They also exposed the implicit, homophone-based expressions that models handle worst.

We formalized 10 offensive and 6 non-offensive categories (16 total), aligned with the six cyber-violence types from the “Network Violence Prevention” report and a Chinese toxicity hierarchy. Each sample is labeled with a binary label (0/1), its specific category, and whether it is a complex expression. Samples come from three sources: 77 real survey utterances, 835 items cleaned from weibo_senti_100k [14], and 1,502 LLM-generated (DeepSeek) supplements that are spread across all three splits. Every sample was labeled by the authors in one round for its binary label, category, and complex flag; we ran no formal agreement study, a limitation we note. Since sentiment differs from offensiveness, weibo_senti_100k serves as a proxy while COLD’s offensive labels are used directly. We then deduplicated, dropped dated expressions, and stratified by length and Chinese-character ratio. Code: github.com/RainbowLightSpirt/SHOUWANG; model: huggingface.co/RainbowLight/tidewatcher-macbert-c2; dataset: huggingface.co/datasets/RainbowLight/between-the-line.

TABLE I. Offensive (10) and non-offensive (6) categories of Between the Line (sample counts in parentheses).

Offensive (10), n = 1,214	Non-offensive (6), n = 1,200
sarcasm & mockery (135)	polite social (200)
manipulation / PUA (121)	opinion & agreement (200)
belittling & preaching (120)	memes & humor (200)
provocation & trolling (120)	daily expression (200)
gender antagonism (120)	sharing & questions (200)
personal threat / curse (120)	boundary sample (200)
profanity & slurs (119)	—
negative emotion (119)	—
discrimination (120)	—
privacy invasion (120)	—

The final dataset contains 2,414 samples: 1,214 offensive (50.3%) vs. 1,200 non-offensive (49.7%), roughly 1.01:1; split 8:1:1 into 1,936/239/239 train/val/test; 480 complex expressions (19.9%), covering both sides (243 offensive, 237 non-offensive). The three datasets (COLD [5], weibo_senti_100k [14], Between the Line) were verified to have zero text overlap, preventing cross-dataset leakage.

VI. Experiments

A. Setup

We use BERT (bert-base-chinese) and MacBERT as base models, and COLD, weibo_senti_100k, and Between the Line as data. We report accuracy, precision, recall, and F1, with recall as the priority metric for the gatekeeper. We split the experiments into three groups that vary data (A), model (B), and training settings (C). A unified script pipeline handles data snapshots, scheduling, training, TensorBoard logging, and evaluation; model selection and hyperparameter scans run on the validation split and are cross-checked with independent seeds, while the test sets (custom and mixed) are held out for final reporting only.

We report four metrics defined on true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN):

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN}) \quad (4)$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \quad (5)$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) \quad (6)$$

$$\text{F1} = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall}) \quad (7)$$

B. A-Group: Data Comparison

Repeated over three seeds, the deployed local gatekeeper reaches F1 0.886 ± 0.003 on the custom test set and F1 0.882 ± 0.005 at about 89% accuracy on the mixed test set (Tables II and IV), distinct from the full system. Adding COLD and weibo jointly improves generalization. Although small, Between the Line clearly boosts recognition on its own style while raising or keeping the other subsets, a sign of useful cross-domain generalization. A gradient scan of weights (8x/15x/25x/50x) shows the sweet spot around 8x–15x; beyond 25x the model over-focuses on the small dataset and overall accuracy falls. A 50x run becomes overly sensitive to negativity and blocks too much, so the 8x weight is used for deployment.

TABLE II. A-group (dataset/weight) results on the mixed test set (17,365 samples). BL = Between the Line.

Run	Training data	Acc	F1
a9	un-tuned baseline	0.452	0.540
a12	weibo+COLD+BL $\times 8$	0.895	0.891
a18	weibo+COLD+BL $\times 15$	0.896	0.891
a17	weibo+COLD+BL $\times 25$	0.895	0.891
a16	weibo+COLD+BL $\times 50$	0.894	0.889

C. B-Group: Model Comparison

With the same data and settings, MacBERT and BERT perform at similar levels (F1 0.896 vs. 0.881), indicating that model choice is a secondary factor. In contrast, fine-tuning brings a much larger gain: untuned MacBERT and BERT score only 0.479 and 0.577, while fine-tuned versions reach 0.896 and 0.881. SnowNLP, a dictionary/rule baseline, scores 0.630 on the custom test set (it reaches only 0.46 on the unified mixed set), confirming the need to replace it with a trained gatekeeper (Table III, Fig. 3).

TABLE III. B-group (model) results on the custom test set (239 samples).

Run	Model	Acc	Prec	Rec	F1
b1	SnowNLP (dictionary)	0.636	0.638	0.622	0.630
b2	BERT (un-finetuned)	0.632	0.674	0.504	0.577
b3	MacBERT (un-finetuned)	0.490	0.487	0.471	0.479
b4	BERT (fine-tuned)	0.883	0.889	0.874	0.881
b5	MacBERT (fine-tuned)	0.895	0.885	0.908	0.896

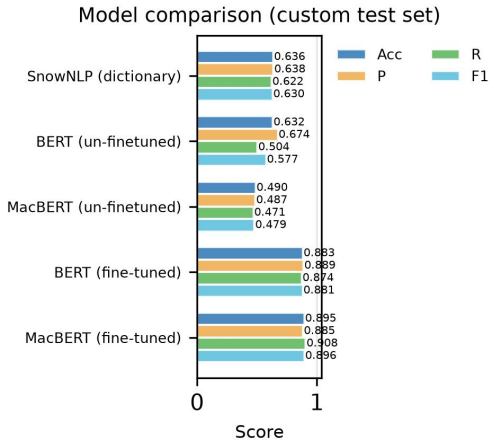


Fig. 3. Model comparison on the custom test set (N = 239): accuracy, precision, recall, and F1.

D. C-Group: Training Settings

Setting $lr = 3e-5$ is the sweet spot; both $1e-5$ and $5e-5$ degrade, with $5e-5$ noticeably hurting precision. At $lr = 3e-5$, raising epochs from 3 to 4 improves F1 from 0.896 to 0.8992, while 5 epochs falls to 0.886 (near overfitting). The optimal configuration is $lr = 3e-5$ with 4 epochs (Table IV, Fig. 4), which is the deployed model. Repeated over three seeds, the deployed configuration ($lr = 3e-5$, 4 epochs, $\times 8$) reaches F1 0.886 ± 0.003 on the custom test set (Table IV); the scan's small gaps (e.g., 0.896 vs. 0.8992) fall within this seed spread and are not claimed as significant.

TABLE IV. C-group (learning rate / epochs) results on the custom test set (239 samples). The deployed configuration (c2) is reported as mean $\pm$ sd over three seeds; the other rows are single-seed and their differences fall within this seed spread.

Run	Config	Acc	Prec	Rec	F1
b5	$lr = 3e-5$, ep 3	0.895	0.885	0.908	0.896
c1	$lr = 1e-5$, ep 3	0.870	0.873	0.866	0.869
c2	$lr = 3e-5$, ep 4	0.886 ± 0.002	0.877 ± 0.006	0.896 ± 0.010	0.886 ± 0.003
c3	$lr = 3e-5$, ep 5	0.887	0.890	0.882	0.886
c4	$lr = 5e-5$, ep 3	0.862	0.847	0.882	0.864

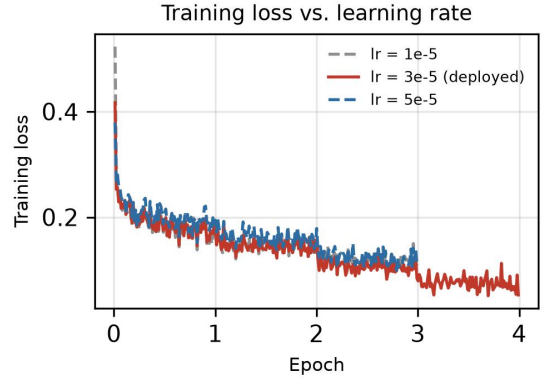


Fig. 4. Training loss versus epoch at different learning rates; the deployed configuration ($lr = 3e-5$, 4 epochs) is highlighted.

E. Case Study: Deployment Choice

Averages do not tell the whole story, so we also inspected actual judgments. The higher-weight run (a18, $\times 15$) is highly sensitive to negativity and blocks nearly every negative statement, including mild venting, whereas the deployed configuration (c2: $\times 8$ weight, $lr = 3e-5$, 4 epochs) screens more stably (Table V). c2 lets mildly upset text through while still catching explicit slurs, so the LLM cold path is only spent on real risk. This is why we deploy the $8\times$ configuration rather than the more trigger-happy alternative.

TABLE V. Example gatekeeper judgments (score / label) of c2 vs. a18.

Text	c2 score	c2	a18 score	a18
it's so annoying that it's raining today	0.249	non-attack	0.997	attack
really angry, don't want to talk anymore	0.399	non-attack	0.989	attack
the service in this shop is really bad	0.627	attack	0.991	attack
nmsl (Chinese net-slang abbreviation)	0.732	attack	0.997	attack

F. System-Level Evaluation

We evaluated the complete pipeline on a 500-item test set (450 emotion/attack items plus 50 factual items) with the deployed implementation, and compared it against a

DeepSeek-only baseline that sends every draft to the LLM. Table VI reports the results.

Escalation is selective: the gatekeeper forwards 94.4% of attack items but only 13.2% of normal items to the LLM. On a balanced stream this is a 53.8% escalation rate; on a realistic stream with 10% attack content it falls to 21.3%, cutting the LLM cost to about 0.14 CNY per 1,000 drafts, roughly 4.3x cheaper than the DeepSeek-only baseline (0.60 CNY per 1,000). Under the strict popup metric the two-stage system is also slightly more accurate (F1 0.911 vs. 0.880), so the gatekeeper buys both lower cost and higher quality.

Latency splits cleanly. The local gatekeeper answers in a median of 126 ms on a laptop CPU, while an escalated draft spends a median of 981 ms in the LLM; end-to-end, escalated drafts take a median of 1,109 ms (P95 1,542 ms), and non-escalated drafts stay near the gatekeeper latency. Because only risky drafts pay the LLM latency, the typical draft is served well under one second. Gatekeeper latency depends on the host CPU; the value here is measured on a standard laptop processor.

TABLE VI. System-level evaluation on 500 items: two-stage pipeline vs. DeepSeek-only baseline.

Metric	Two-stage	DeepSeek-only
Strict F1 (popup = attack)	0.911	0.880
Accuracy / Precision / Recall	0.918 / 0.990 / 0.844	0.882 / 0.895 / 0.867
LLM calls (500 items)	269	500
LLM latency, median (ms)	981	932
End-to-end latency, median (ms)	795	932
API cost (CNY / 1,000 drafts)	0.355	0.600

VII. Conclusion and Future Work

We presented Shouwang, a fast-slow collaborative intervention system that guides users before they post instead of punishing them afterward. The three comparative experiments show that data matching real expressions is the main lever of detection quality, while model choice and training settings matter less. The selected MacBERT gatekeeper (lr 3e-5, 4 epochs, 8× weight) scores F1 0.886 ± 0.003 over three seeds on our custom test set and is ready to deploy.

Limitations remain. The intervention effect needs large-scale live user studies, and the single-seed runs should be repeated. Part of the data is LLM-generated and may carry stylistic artifacts; with single-round labeling and no agreement study, we do not overclaim dataset quality. Drafts are screened on-device and only escalated ones reach the third-party LLM; a gatekeeper-only local mode is possible but not implemented, and drafts are not retained. Sarcasm, group-directed discrimination, and phrase-level detection stay open.

References

[1] D. Xiong, “Research on structured analysis method of e-commerce user reviews based on large language model,” *E-Commerce Letters*, vol. 15, no. 3, pp. 970–979, 2026.
[2] D. Li, Y. Wang, and Q. Zhai, “A deep learning-based text sentiment analysis method for web comments,” *Modeling and Simulation*, vol. 13, no. 5, pp. 5372–5381, 2024.

[3] X. Jin, L. Li, and L. Zhong, “Review spam detection approach based on topic model and sentiment analysis,” *Computer Science*, vol. 44, no. 10, pp. 254–258, 2017.
[4] isnowfy, “SnowNLP: Simplified Chinese text processing.” [Online]. Available: <https://github.com/isnowfy/snownlp>
[5] J. Deng et al., “COLD: A benchmark for Chinese offensive language detection,” in *Proc. EMNLP*, 2022, pp. 11580–11599.
[6] J. Lu, B. Xu, X. Zhang, C. Min, L. Yang, and H. Lin, “Facilitating fine-grained detection of Chinese toxic language: Hierarchical taxonomy, resources, and benchmarks,” in *Proc. ACL*, 2023, pp. 16235–16250.
[7] A. Jiang, X. Yang, Y. Liu, and A. Zubiaga, “SWSR: A Chinese dataset and lexicon for online sexism detection,” *Online Social Networks and Media*, vol. 27, art. no. 100182, 2022.
[8] T. Davidson, D. Warmesley, M. Macy, and I. Weber, “Automated hate speech detection and the problem of offensive language,” in *Proc. ICWSM*, 2017, pp. 512–515.
[9] M. Mozafari, R. Farahbakhsh, and N. Crespi, “Hate speech detection and racial bias mitigation in social media based on BERT model,” *PLOS ONE*, vol. 15, no. 8, e0237861, 2020.
[10] D. Kumar, Y. AbuHashem, and Z. Durumeric, “Watch your language: Investigating content moderation with large language models,” in *Proc. ICWSM*, 2024.
[11] R. Gorwa, R. Binns, and C. Katzenbach, “Algorithmic content moderation: Technical and political challenges in the automation of platform governance,” *Big Data & Society*, vol. 7, no. 1, 2020.
[12] Y. Cui, W. Che, T. Liu, B. Qin, S. Wang, and G. Hu, “Revisiting pre-trained models for Chinese natural language processing,” in *Findings of EMNLP*, 2020, pp. 657–668.
[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
[14] weibo_senti_100k: Sina Weibo sentiment dataset. Hugging Face. [Online]. Available: https://huggingface.co/datasets/dirtycomputer/weibo_senti_100k